

ITI5202 – DATA PROCESSING FOR BIG DATA

ASSESSMENT 2 PHASE 2 – PROJECT REPORT

Project Title:

**Predicting Used Cars Price in the US
based on Its Condition and Features**

Student:

- 1. Eddy (33494068)**
- 2. Fariz Akyas (33599890)**
- 3. Robby L. Aurelio (32889356)**

1. Project Description

1.1. Background

Many people may think that owning a car is just a secondary or even tertiary need. On the contrary, in the US, it is crucial to own a car because the community and infrastructure depend on cars as the primary means of traveling or commuting. This statement is based on an article written by Inkstick Media which states that “Car ownership has become a de facto prerequisite for Americans to access the labor market and engage in civic life” ^[1]. Nonetheless, not everyone can buy a car as car prices can be relatively expensive. That is why, some people may prefer to buy used cars instead of new cars, which are cheaper compared to brand-new cars.

Sadly, due to the adverse effect of the COVID-19 pandemic on the economic sector, used car prices have skyrocketed in recent years (can be seen in Figure 1). The main cause for this problem is that the production of new cars has mostly ceased due to the lack of semiconductor chips available ^[2]. This creates a problem for people who still haven’t owned a car yet or their car is broken as it becomes harder for them to acquire a new car. A woman in an interview with the New York Times, Ms. Cote, 29, said “Food’s gone up a bit, but the cost of housing, health care, and cars is pretty brutal.” Because of the rise in car prices, she became unable to make any savings because all of her income was spent on her necessities ^[3].

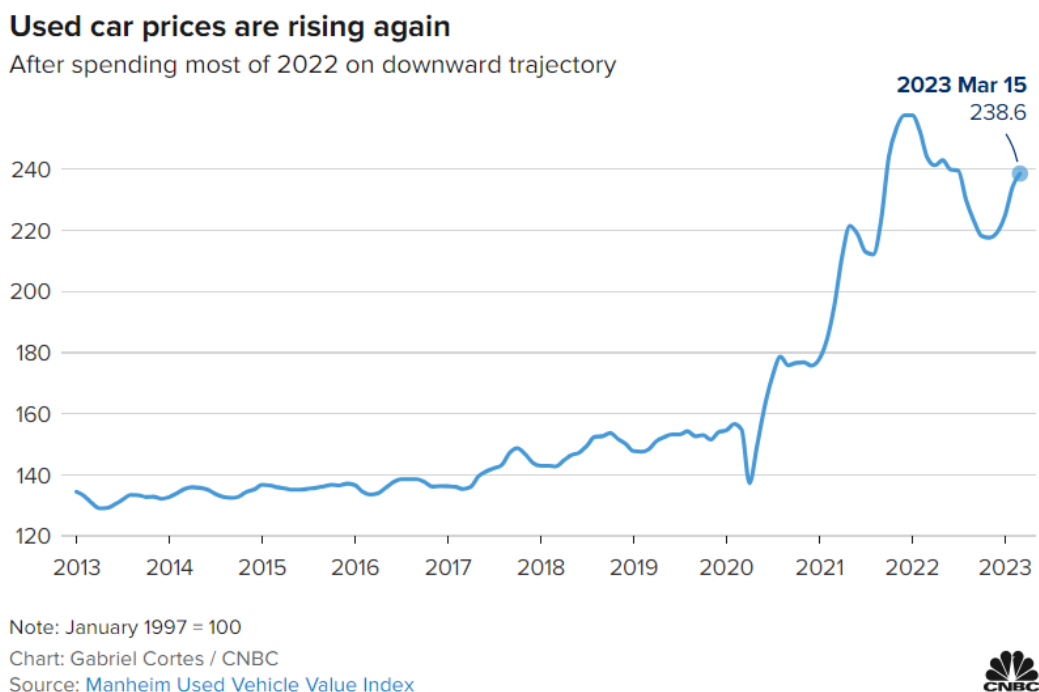


Figure 1. Used cars price trend in the US ^[2]

1.2. Problem Formulation

Because the rise in used car prices was caused by the COVID-19 pandemic, it is only natural to expect that their price will be back to normal after the pandemic passes. Nevertheless, in an interview with CNBC, Charlie Chesbrough, senior economist and senior director of Industry Insights at Cox Automotive said “There is still a lot of demand out there that just has not been met yet. It’s unsatiated demand. Until the new market can rebuild such that demand is met and that there is enough product out there for everybody, the U.S. markets are going to continue to be doing very robust sales”. Without new vehicles, there are no used vehicles. Therefore, in the worst-case scenario, it is even possible that the prices may rise again by 60% like when COVID-19 happened ^[4].

To make matters worse, nowadays people prefer to buy cars online without even going to the dealer and inspect the car themselves before buying it beforehand. Based on the data from AA Cars, one-in-five (21%) people bought a car without seeing it first in 2022 compared to 2021, which was only just one-in-ten (10%) ^[5]. This phenomenon gives rise to a problem as the cars bought might not be the same as what they had in mind, thus it is more likely for the dealer to outsmart the buyer this way. Although this can be mitigated by reading online articles that advise about how to buy used cars, it would be better if there is a way to know the correct prices of the car reliably. Thus, it may help the buyer to be able to make good purchases. One way of doing this is by creating a model to predict the used car prices based on the car’s condition using machine learning such as linear regression.

1.3. Past Research

Thakkar et al. (2023) have researched predicting car prices using linear regression, which is considered to be a popular way to do it ^[6]. From their research, they found that linear regression is indeed suitable for predicting car prices with an accuracy ranging from 78% - 86.5%. In addition, they also stated that typically, the models are made by using the car’s manufacture, kms_driven, and condition as the predictors, and some researchers even do feature engineering on the predictors to improve the prediction. Overall, they said that “the model architecture for old car price prediction using linear regression is relatively straightforward, with the primary focus on selecting appropriate input variables and optimizing the coefficients through the training process” ^[6].

Other than Thakkar et al. (2023), Mammadov (2021) also researched predicting US car prices using linear regression. He was able to get a 0.9 R^2 value which means that linear regression is really accurate to predict car prices ^[7]. In addition, the linear regression method has also been used to predict used car prices in other countries as well, such as Indonesia. Puteri and Safitri (2020) used multiple linear regression to predict car prices and were able to find out that the age of the cars is the most important feature with a trust value of 63.2% ^[8].

One crucial difference between the previous research and our proposed project is that they did not consider using a lot of features as predictors as we managed to obtain the dataset with a lot of attributes. When using a lot of features as predictors, there is a high risk of overfitting, which may impact the model's performance. One approach to handle it is by applying regularization techniques like what Demir-Kavuk et. al. (2011) did with their research on biological prediction problems. From their research, they were able to reduce 6,000 features to only 50 using the L1 regularization (LASSO) and found that the L2 regularization (Ridge) increased their prediction's accuracy^[9]. Therefore, it can be concluded that the regularization technique can help to reduce the number of predictors (to prevent overfitting) and further enhance prediction accuracy.

1.4. Objective

The objective of this research is to build a machine-learning model that is accurate and easy to interpret while selecting features that are important for the predictions. The dataset that this project is going to use is from the used car data set in the US that has 3 million records and 66 attributes which makes it larger than the other used car price predictions research that was outlined before. Due to this, other than the normal linear regression (ordinary least squared), regularization techniques like the LASSO and Ridge techniques are also going to be tested to see if they can give a better performance.

2. Business Model

2.1. Business Area and Benefits

The sector that will benefit the most from this research is the automotive industry, especially the used car market. Moreover, the benefit will not only be limited to just the dealers but also the potential buyers and even the car owners who want to sell their cars. The resulting linear regression model can be used to determine the price of the car fairly based on its features and condition. This price then can be used as a benchmark for the seller to determine the initial price or even the buyers can use this price to negotiate if the initial price is too expensive. In addition, the feature analysis done will help to determine which factor has a great impact on the price of the car by seeing its variable importance. With this knowledge, car owners who potentially sell their cars in the future can know which feature or condition of the car they need to keep well so that the price stays high.

2.2. Stakeholders

- Used car dealers:
Dealers can use the resulting model to determine the price of the car that they are going to sell. By using the calculated price as a benchmark, the dealer can safely decide on the price without “damaging” the used car market by setting the price too low or too high.

- **Car consultants:**
Car/automobile consultants are people commissioned by either dealers or buyers to provide information necessary for them to deal with the cars they are interested in. Based on the feature analysis result, the consultant can also use it to guide the buyers about what features or specifications are important.
- **Potential buyers:**
The potential buyers can easily check whether the price given on the car that they want to buy is following the market's price or not by the car's features and conditions. This is done to avoid potential scamming from the seller by setting the price too high.
- **Car owners who want to sell their car:**
For car owners, the usage of the model is a combination between dealers and buyers. They can use the model to set the price appropriately and if they sell it to a third party (dealer), they can know whether the price set by them is suitable or not.

2.3. Challenges

Challenges	Mitigation
The raw dataset's size is too big (± 3 million rows). This may result in a slower processing speed due to the large data.	Apply parallel processing by partitioning the data.
Some of the fields (columns) do not have a proper data dictionary.	Discard the columns that are hard to guess. Use assumptions on the clear ones.
The dataset has a large number of missing values. However, imputing specific values on some columns is not possible as each car has its own conditions or features.	If the data type is Boolean (True/False), we can just impute False as the default value. On the other hand, for other columns, we can just drop the missing values as the remaining records should be enough for training a model considering the size of the dataset.

3. Data

3.1. Data Source and Characteristic

The dataset we used was obtained from Kaggle (kaggle.com/datasets/ananyamital/us-used-cars-dataset/data) and was downloaded on October 10, 2023. This dataset contains around **3 million** used car sales records from the US. The author of the dataset, Ananay Mital, acknowledged that they obtained the data with a self-made crawler on Cargurus inventory in September 2020. The dataset consists of 3 million rows and 66 columns. The data dictionary is provided in Table 1. However, because we decided to not use all columns, we will only list the metadata of those selected columns.

Table 1. Data dictionary of the used cars dataset

Column Name	Type	Description
vin	String	Vehicle Identification Number is a unique encoded string for every vehicle.
back_legroom	String	Legroom in the rear seat.
body_type	String	Body Type of the vehicle. Like Convertible, Hatchback, Sedan, etc.
city_fuel_economy	Float	Fuel economy in city traffic in km per litre.
daysonmarket	Integer	Days since the vehicle was first listed on the website.
engine_cylinders	String	The engine configuration. Eg: I4, V6, etc.
engine_displacement	Float	The measure of the cylinder volume swept by all of the pistons of a piston engine, excluding the combustion chambers.
franchise_make	String	The company that owns the franchise.
front_legroom	String	The legroom is in inches for the passenger seat.
fuel_tank_volume	String	Fuel tank's filling capacity in gallons.
fuel_type	String	Dominant type of fuel ingested by the vehicle.
has_accidents	Boolean	Whether the vin has any accidents registered.
height	String	Height of the vehicle in inches.
highway_fuel_economy	Float	Fuel economy in highway traffic in km per litre.
horsepower	Float	Horsepower is the power produced by an engine.
isCab	Boolean	Whether the vehicle was previously taxi/cab.
is_cpo	Boolean	Pre-owned cars certified by the dealer. Certified vehicles come with a manufacturer warranty for free repairs for a certain time period.
is_oemcpo	Boolean	Pre-owned cars certified by the manufacturer.
length	String	Length of the vehicle in inches
listing_color	String	Dominant color group from the exterior color.
mileage	Integer	-
owner_count	Integer	-

Column Name	Type	Description
price	Float	-
torque	String	-
transmission	String	-
wheel_system	String	-
wheelbase	String	-
width	String	-
year	Integer	-

3.2. Data Wrangling

Before we can use the dataset for analysis purposes, we need to clean the data first. First, to lessen the processing burden, we can create a subset of the data by only taking the necessary columns. Henceforward, the wrangling process will only be done on the subset.

As mentioned in the previous section, the dataset contains many missing values that need to be dealt with before using it for training a model. In our case, if the column type is Boolean/logical (True/False), such as `is_new`, `is_cpo`, `is_oemcpo`, `has_accidents`, and `isCab`, we decided to replace the NAs with a "False" value. The reason is that, for the `has_accidents` and `isCab` columns, naturally the default value is False. In contrast, for the other three, we assume that missing values happen because the dealer deliberately did not list those details to hide the actual age of the car or that the car did not have certification. On the other hand, for the other columns, we decided to just remove the rows that contain NAs in those columns. The reasoning behind this is that the car itself should have its own unique feature or condition, thus imputing the median or mode may distort the truth about the data. It is possible to do in-depth investigating to predict the actual value based on the other known columns. However, for this initial stage, we decided that it is not necessary as the volume of the data after removal is still big enough for training a machine learning model. The comparison of the size between the original dataset and the subset (after dealing with missing values) is provided in Figure 2.

```
> dim(cars)
[1] 3000040      66
> dim(wrangled_cars)
[1] 704408      32
```

Figure 2. Comparison of the size of the original dataset (above) and the subset (below)

Note: due to some difficulties when using PySpark in the Jupyter Notebook, the above wrangling process was done using R in RStudio. The "semi-cleaned" dataset was then written to a new CSV file that will be further processed using PySpark in Jupyter Notebook.

After dealing with missing data, the next step is to inspect the structure of the resulting data frame. We noticed that some of the columns have incorrect data types, which are listed as a string but should be numeric (integer or double). This happens because those columns contain not only the values but also their unit of measurement, such as “in” or “gal”, which are non-number characters. To convert those columns to numeric type (here we choose to convert them to double), we need to remove the non-number characters first. Then, after the values only contain numbers, we can safely convert the column type into a double type. It turns out that the dataset still contains some missing values (listed as “--“, not empty). Thus, after converting the columns to numeric, those values will turn to NA. Therefore, those records need to be removed. The comparison between the initial schema and after wrangling is provided in Figure 3.

<pre>(704408, 32) root -- vin: string (nullable = true) -- back_legroom: string (nullable = true) -- body_type: string (nullable = true) -- city_fuel_economy: integer (nullable = true) -- daysonmarket: integer (nullable = true) -- engine_cylinders: string (nullable = true) -- engine_displacement: integer (nullable = true) -- franchise_make: string (nullable = true) -- front_legroom: string (nullable = true) -- fuel_tank_volume: string (nullable = true) -- fuel_type: string (nullable = true) -- has_accidents: boolean (nullable = true) -- height: string (nullable = true) -- highway_fuel_economy: integer (nullable = true) -- horsepower: integer (nullable = true) -- price: double (nullable = true) -- width: string (nullable = true) -- year: integer (nullable = true) -- is_cpo: boolean (nullable = true) -- is_oemcpo: boolean (nullable = true) -- is_new: boolean (nullable = true) -- listing_color: string (nullable = true) -- mileage: integer (nullable = true) -- owner_count: integer (nullable = true) -- power: string (nullable = true) -- torque: string (nullable = true) -- isCab: boolean (nullable = true) -- transmission: string (nullable = true) -- wheel_system: string (nullable = true) -- length: string (nullable = true) -- wheelbase: string (nullable = true) -- model_name: string (nullable = true)</pre>	<pre>(694605, 30) root -- vin: string (nullable = true) -- back_legroom: double (nullable = true) -- body_type: string (nullable = true) -- city_fuel_economy: integer (nullable = true) -- daysonmarket: integer (nullable = true) -- engine_cylinders: string (nullable = true) -- engine_displacement: integer (nullable = true) -- franchise_make: string (nullable = true) -- front_legroom: double (nullable = true) -- fuel_tank_volume: double (nullable = true) -- fuel_type: string (nullable = true) -- has_accidents: boolean (nullable = true) -- height: double (nullable = true) -- highway_fuel_economy: integer (nullable = true) -- horsepower: integer (nullable = true) -- price: double (nullable = true) -- width: double (nullable = true) -- year: integer (nullable = true) -- is_cpo: boolean (nullable = true) -- is_oemcpo: boolean (nullable = true) -- is_new: boolean (nullable = true) -- listing_color: string (nullable = true) -- mileage: integer (nullable = true) -- owner_count: integer (nullable = true) -- torque: double (nullable = true) -- isCab: boolean (nullable = true) -- transmission: string (nullable = true) -- wheel_system: string (nullable = true) -- length: double (nullable = true) -- wheelbase: double (nullable = true)</pre>
---	--

Figure 3. The comparison of the original schema and the corrected one

3.3. Exploratory Data Analysis

After the data is cleaned enough, the next step is to do some exploratory analysis by investigating the basic statistic summary or the data distribution. From the schema in Figure 3, it can be seen that there are three main types of data, numeric (integer and double),

string, and Boolean (logical). Thus for each type, we will give different treatments about how to explore the data.

For the numerical data, we decided to just show a summary table containing the basic descriptive statistics, such as mean, median, standard deviation, first quartile, third quartile, minimum, and maximum, about each column. The result can be seen in Figure 4. By comparing the values of the mean and median of each column, we can get a basic understanding of the data distribution. More or less the distribution looks like this:

- The back_legroom, front_legroom, and width columns are heavily skewed to the left.
- The year, wheel_base, fuel_tank_volume, height, and length are skewed to the left.
- daysonmarket, engine_displacement, price, and mileage are skewed to the right.
- city_fuel_economy, highway_fuel_economy, horsepower, and torque have a normal distribution.
- owner_count is a unique case as the minimum, first quartile, median, mean, and third quartile have the same value (around 1), whereas the maximum is much larger. This may happen due to a large number of records having a value of 1, thus heavily skewed the distribution.

summary	back_legroom	city_fuel_economy	daysonmarket	engine_displacement	front_legroom
count	694605	694605	694605	694605	694605
mean	354.2146658892464	21.061470907926086	43.39215525370534	2952.6951288862015	369.37921120636906
stddev	89.56128155576003	4.5803941584326	73.20664937370847	1144.8867181940902	132.27516507171316
min	0.0	9	0	1000	0.0
25%	356.0	17	11	2000	408.0
50%	376.0	21	25	2500	416.0
75%	394.0	25	48	3600	428.0
max	598.0	74	2688	8100	525.0

summary	fuel_tank_volume	height	highway_fuel_economy	horsepower	price	width
count	694605	694605	694605	694605	694605	694605
mean	125.25643351257189	589.1953354784374	28.518294570295346	247.61337018881235	24024.255515911922	683.1355230670669
stddev	75.18873606030637	188.33768818350674	5.538200724878552	82.07694883453541	12494.583131147989	253.47022643308836
min	8.0	49.0	11	55	484.0	66.0
25%	26.0	573.0	24	178	16100.0	709.0
50%	153.0	653.0	28	245	21500.0	744.0
75%	185.0	693.0	33	295	29993.0	818.0
max	335.0	1086.0	75	808	2698500.0	986.0

summary	year	mileage	owner_count	length	wheelbase	torque
count	694605	694605	694605	694605	694605	694605
mean	2016.7401847092951	46099.33506381325	1.3231491279216245	1750.6762404532071	1036.8719804781135	251.07505272780932
stddev	2.684279766487007	36204.183345530495	0.656330773502176	526.3323986198924	303.10857424813423	90.1439708562333
min	1987	0	1	152.0	97.0	58.0
25%	2016	22709	1	1793.0	1051.0	177.0
50%	2017	35496	1	1884.0	1093.0	258.0
75%	2019	58879	1	1962.0	1142.0	275.0
max	2021	399496	12	2661.0	1646.0	811.0

Figure 4. Summary statistics of the numerical columns in the dataset

Other than just investigating the basic descriptive statistics, we also made a boxplot and histogram of the 'price' column to dig deeper into the distribution of the data. The column 'price' is chosen because it will be used as the target variable for our machine learning model, thus it is better to explore more about the column. The boxplot and histogram can be seen in Figure 5 and Figure 6. From the boxplot, it can be seen that the 'price' column has a large number of outliers, especially the ones that are greater than the upper boundary. This causes the distribution of the 'price' in the histogram to look heavily skewed to the right, even though the mean and median should not be that significantly different.

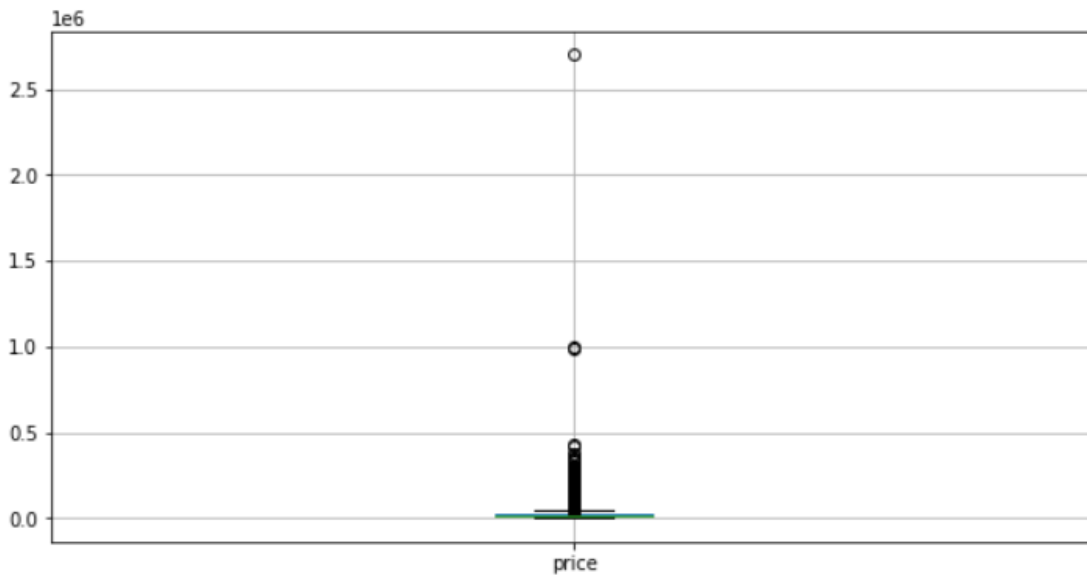


Figure 5. Boxplot of the 'price' column



Figure 6. Histogram of the 'price' column

Then, for the string data type, we only count the frequency (number of occurrences) of each unique value (category) in the column. The result can be seen in Figure 7.

body_type	count	franchise_make	count	fuel_type	count
SUV / Crossover	347161	Chevrolet	97777	Gasoline	654346
Sedan	197005	Ford	94944	Flex Fuel Vehicle	35978
Pickup Truck	74484	Jeep	63578	Diesel	3175
Minivan	24991	Toyota	57077	Biodiesel	686
Coupe	16797	Honda	53988	Hybrid	413
Hatchback	14184	Nissan	40724	Compressed Natura...	7
Wagon	14154	Kia	26629		
Convertible	5072	Hyundai	24885		
Van	757	Buick	23940		
		Subaru	21002		
		Volkswagen	18797		
		BMW	18765		
		Mercedes-Benz	15134		
		Dodge	14472		
		Mazda	14073		
		GMC	11360		
		Lexus	10941		
		Cadillac	10163		
		Acura	9595		
		Audi	8577		

only showing top 20 rows

listing_color	count	transmission	count	wheel_system	count
WHITE	142196	A	570802	FWD	311793
BLACK	140293	CVT	108126	AWD	191167
SILVER	97855	M	13242	4WD	123943
GRAY	90852	Dual Clutch	2435	RWD	43546
UNKNOWN	87690			4X2	24156
RED	61732				
BLUE	58146				
BROWN	4800				
GREEN	4534				
ORANGE	2310				
GOLD	1779				
TEAL	1070				
YELLOW	868				
PURPLE	444				
PINK	36				

Figure 7. Frequency of each unique value (category) of the string column in the dataset

Lastly, for the Boolean columns, we make a pie chart to compare the percentage of True and False values of each column. The collection of the pie charts can be seen in Figure 8. From those pie charts, the False records dominate every column. This may happen because here we set the False value as the default value to be imputed to the records with a missing value. Thus this may skew the actual data distribution.

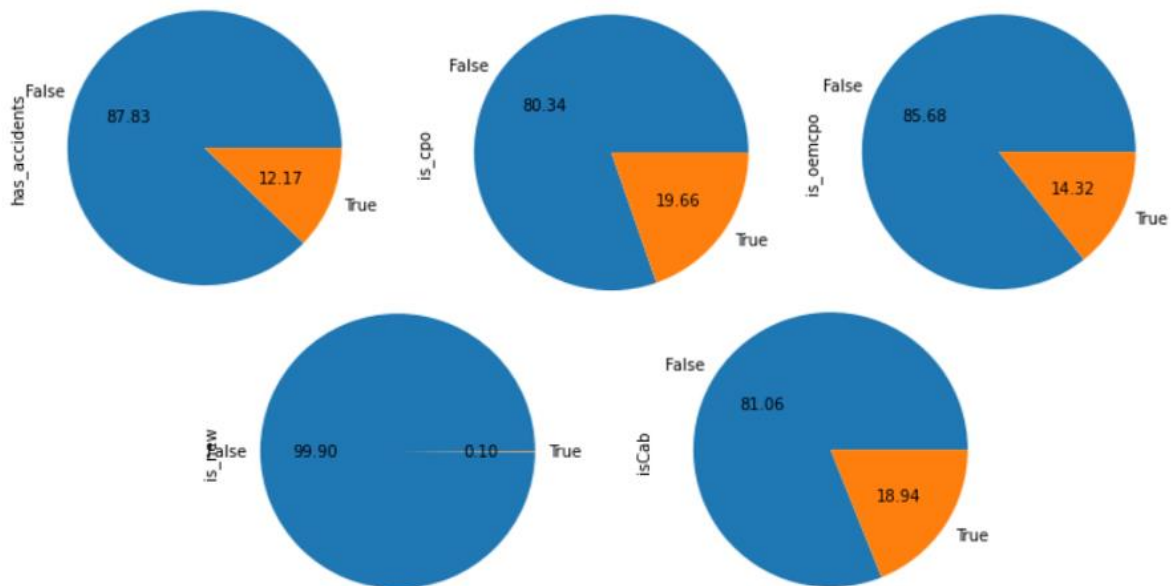


Figure 8. A collection of the pie chart to show the distribution of the Boolean columns

4. Machine Learning

To solve the problem proposed in the previous section, we can transform it into a regression problem with the price as the target variable and the other columns as the predictors. The type of regression we chose is the **Linear Regression Model** as from past research, it is proven to be suitable for our case and one of the best practice. In addition, to avoid the probability of overfitting, we also try to apply the regularization method (which will be explained in detail in a later section). Therefore, we will create three types of models: (1) **normal linear regression model (least squared method)**, (2) **LASSO model**, and (3) **Ridge model**. The overall process done in our project can be seen in Figure 9.

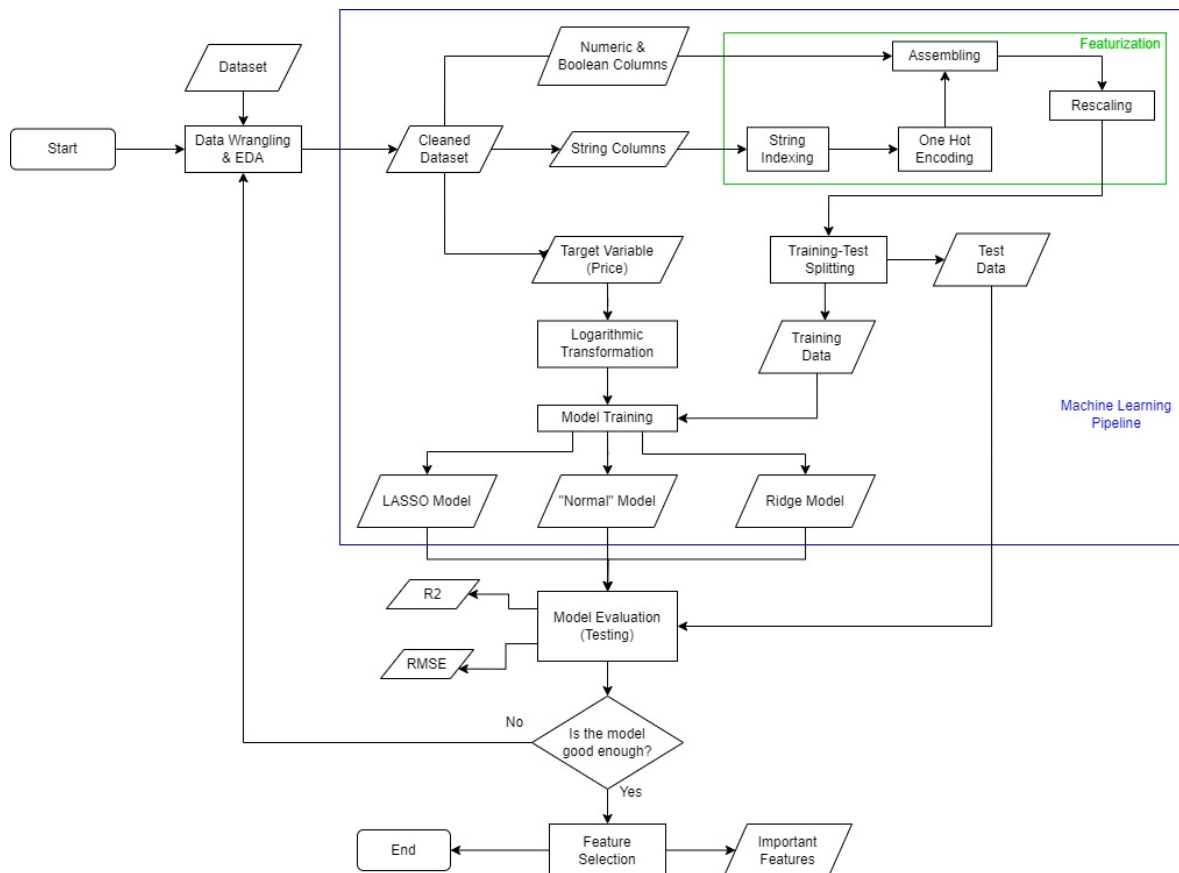


Figure 9. The overall process of the project

4.1. Data Preparation (Featurization)

Before using the data to train a model, the data needs to be pre-processed first. The main reason is that some of the predictors are string values, whereas Spark's machine-learning model can only take a numeric vector as the input. Therefore, we need to do some transformation, which is called featurization. The featurization process we used consists of three steps:

a. String Indexing:

A process to convert the categorical string values into category indices which are ordered by their frequencies. The most frequent label will be given the index 0, then the next most frequent label will be 1, and so on. By using the string indexer, the categories, which were strings, can be converted into numerical values. In PySpark, this process can be done by using the *StringIndexer* function from the "pyspark.ml.feature" library.

b. One-Hot Encoding:

A process that maps a column of category indices to a column of binary vectors with at most a single one-value per row. This conversion or pivoting is done to avoid the problem of ordinality caused by the string indices. For example, the index 2 does not mean that it is twice the value of the index 1. That is why, by using the one-hot Encoding, the category indices are converted to vectors containing binary values. In PySpark, this process can be done by using the *OneHotEncoder* function from the “pyspark.ml.feature” library.

c. Assembling:

A process that assembles multiple columns into a vector column which will be used as the predictor or the feature for the model. For this function, it only accepts numeric, Boolean, and vector types. That is why all the different string data types need to be transformed into numeric vectors first. In PySpark, this process can be done by using the *VectorAssembler* function from the “pyspark.ml.feature” library.

After doing the three processes described above, the assembled vector of the predictors will only contain numeric values, which consist of the original numeric variables and the binary values from the transformed string variables and Boolean values (‘True’ is treated as 1 and ‘False’ is treated as 0). In contrast with the binary values, the numeric values have a wide range of values, in which each variable has a different distribution. Therefore, it is better to rescale those numeric values into a common value, which is between 0 and 1 (same as the binary values). The rescaling can be done by using this formula:

$$rescaled(x) = \frac{x - \min(x)}{\max(x) - \min(x)}$$

where $\min(x)$ and $\max(x)$ are the minimum and the maximum value of the variable, respectively. This can be done easily in PySpark by using the *MinMaxScaler* function from the “pyspark.ml.feature” library.

In addition, because all of the predictors’ values were rescaled between 0 and 1, it is also better to transform the target variable, ‘price’, into smaller values. This will also help the feature selection process later on as it will encourage the coefficients to be small enough to be near zero or even zero. Here, we apply a logarithmic transformation with base 10 to make the ‘price’ values small enough to be near 0 and 1.

4.2. Building the Machine Learning Model

The machine learning model that this project will use is linear regression. Linear regression assumes a linear relationship between the independent and dependent variables which also correspond with our case. Moreover, based on past research, linear regression is proven to be one of the best practices when doing a used car’s price prediction. In PySpark, a linear

regression model can be easily built by using the *LinearRegression* function from the “pyspark.ml.regression” library.

While linear regression is regarded as one of the most powerful machine learning models, it suffers from two flaws which are:

- It's prone to overfitting when there are many attributes.
- It cannot easily express a non-linear relationship.

A way to solve these two flaws is by increasing the complexity of the models but increasing model complexity will also increase the risk of overfitting. This can be dealt with by applying regularization. Regularization is a way to control the complexity of a linear regression model by penalizing the coefficients that are not relevant to the prediction. It is done by introducing a constraint that serves as a penalty to prevent overfitting, but in exchange, it trades off increasing bias for lowering variance.

There are two types of regularization techniques used in machine learning, which are LASSO and Ridge. The main difference between LASSO and the Ridge model is how they introduce the penalty. For the LASSO model, the weight ‘w’ is the absolute value, while the Ridge model uses the squared weight. This difference will be clearer by seeing the formula:

- LASSO: $Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N |w_i|$
- Ridge: $Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N (w_i)^2$

For our project, we will try to compare the resulting model by applying the regularization methods (both the LASSO and the ridge model) and no regularization. In PySpark, this can be easily done by tweaking the parameters of the linear regression model in the ‘MLlib’ library. For the with regularization model, we can set the ‘elasticNetParam’ value equal to 1 for the LASSO model or 0 for the ridge model, whereas for the no regularization model, we specify nothing.

Other than specifying the type of regularization, we also need to specify the ‘regParam’, which is noted as λ in the model. Lambda (λ), also called the Regularization Rate, is the amount of regularization that is applied to the model which should be inserted manually. Based on its value, it will determine how big of a penalty the regularization gives. When choosing its value, it must strike a balance between simplicity and fitness of the model. If we increase the λ , the model will be simpler and can reduce the variance, but if it's too high it will force the important attributes also to 0 therefore increasing the risk of underfitting. In contrast, if it is too low, the model will be too complex and increases the risk of overfitting.

On the other hand, for the normal linear regression with no regularization, the λ is set to 0. By setting it to 0, the calculation for both LASSO and Ridge will be 0 and the penalty coefficient will be 0. Hence, the calculation corresponds to normal linear regression calculation, which is the ordinary least squares.

To be able to determine the best value of λ for our model, we will also use the k-fold cross-validation method. Cross-validation is a resampling method that shuffles part of the data to test and train a model repeatedly on different iterations. Its main use is to estimate how accurately the predictive model will perform and for hyperparameter tuning to choose the optimal parameters. Here, we apply four different values of λ (0.001, 0.01, 0.01, and 1.0) and let the algorithm determine which is the best value of λ for our model.

4.3. Model Testing

After the three models are made, the models need to be tested on the test data. To check the variance of the model and to determine whether the model is overfitting or not, the test data has been randomly split into 10 different subsets. Here, we use the RMSE and R^2 as the model evaluators.

RMSE (Root Mean Square Error) calculates the difference between a statistical model prediction and the actual values. It is the standard deviation of the residual which is the distance between the regression line and the data points. The lower this value is the better.

The formula for calculating the RMSE:

$$RMSE = \sqrt{\frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{N}}$$

Where N = the number of data, y_i = the actual value, and $\hat{y}_i$ = the predicted value.

On the other hand, R^2 represents the strength of the relationship between the model and the target variable. The value of R^2 ranges from 0-1 with the value closer to 1 means the model is more accurate.

The formula for calculating the R^2 :

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}, \quad SS_{tot} = \sum_{i=1}^N (y_i - \bar{y})^2, \quad SS_{res} = \sum_{i=1}^N (y_i - \hat{y})^2$$

Where N = the number of data, y_i = the actual value, $\bar{y}$ = the mean value, and $\hat{y}_i$ = the predicted value.

Both evaluations can be done easily in PySpark by using the *RegressionEvaluator* function from the “pyspark.ml.evaluation” library and specify the *metricName* argument to “rmse” or “r2” depending on the wanted metric.

4.4. Feature Selection

After evaluating the model, we can also do a feature selection by investigating the importance of each feature. Here, we evaluate the importance by seeing the coefficient. Because we also have rescaled the data to a common scale, which is between 0 and 1, the coefficient will not be affected by the range of the data, i.e. the features with “big number” will have smaller coefficients, and vice versa. By seeing the coefficients, it can be implied that the features with a coefficient of near zero are not important when doing the prediction, whereas the features with a coefficient of zero are explicitly not used in the prediction.

5. Result and Discussion

5.1. Parallelism

In this project, we have set the parallelism parameter to 2 when initiating the cross-validation for each model. This means the process will be parallelized into 2 processors. The results of the parallelism done when training each model are provided in Figure 10 to Figure 12. The diagrams were obtained from the Spark Web UI's event timeline.



Figure 10. The event timeline for training the no-regularization model



Figure 11. The event timeline for training the LASSO model

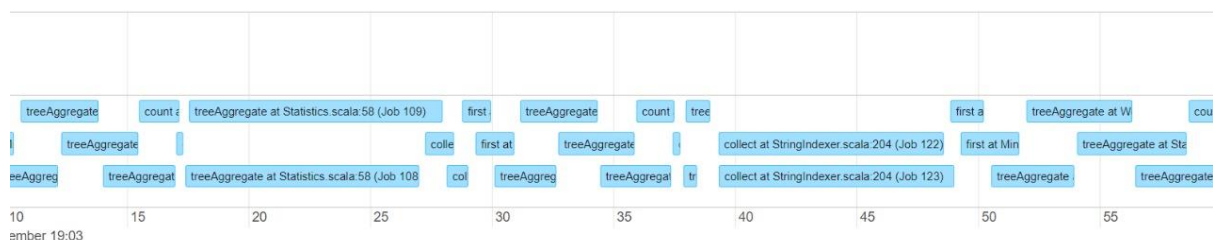


Figure 12. The event timeline for training the Ridge model

From the timelines above we can see that even with the parallelism parameter set to 2, most of the code is still processed in serial matter, especially from the no-regularization model. However, for the LASSO and Ridge model, it can be seen that some of the processes were parallelized into 2 processors. This parallelized is shown with two block processes on the same timeline.

5.2. Model Evaluation

The result of the RMSE and R^2 calculations is provided in Figure 13 and Figure 14.

lasso	reg	ridge
4739.265707058171	4625.039167369196	4621.462722564097
4988.899534200487	4853.560004437842	4853.77279993445
5032.832293203613	4864.532871276712	4867.892605139077
4813.180185489223	4665.05973535641	4664.093001666336
15159.436028017639	15111.842388241183	15112.224154259262
5058.670681319768	4998.546577095574	4994.442154561764
4707.233223717902	4605.791944564405	4603.125513300024
4822.3002831557915	4699.3868102033775	4700.168956059335
5011.375872310053	4853.285114124666	4855.152466972585
6999.179762568387	6948.685168689089	6947.5427683453

Figure 13. Result of the RMSE calculations of each model for each subset of test data

lasso	reg	ridge
0.8417185516718126	0.8492564532859959	0.849489496729607
0.8337400260022031	0.8426383147217895	0.8426245159464583
0.8268993432393588	0.8382828485930234	0.8380593886099934
0.8383063494190678	0.8481051167784697	0.8481680641765184
0.3518287100317383	0.35589223921294166	0.3558596949930163
0.8205133578163881	0.8247545286702365	0.8250422067546712
0.8352351630381165	0.8422600379254976	0.8424426258590356
0.8316154942447636	0.8400898564287924	0.8400366224774349
0.8257005255865064	0.8365241015391632	0.8363982791715222
0.6958693022215394	0.7002416890709401	0.7003402446506938

Figure 14. Result of the R^2 calculations of each model for each subset of test data

From the result, it can be seen that overall, there is no significant difference between the three models and also between the subsets, except in subset numbers 5 and 10. After investigation, it was found that the subset numbers 5 and 10 contain an extreme outlier in which the price is around or more than 1 million USD, whereas the other prices are around or less than 500,000 USD. Even though the training data also contain an extreme outlier, it is only 1 data

point which may not affect the prediction much. Therefore, it is likely that the model cannot predict an extremely high price.

After removing the extreme outliers from the test data numbers 5 and 10, we tested them again and found out that both the RMSE and R^2 were not significantly different from the other subsets (the new results are provided in Figure 15 and Figure 16). Therefore, we can conclude that all three models are not overfitted because the evaluation results between each subset of the test data have similar results (the variance is low).

	lasso	reg	ridge
	4789.668140863779	4640.223231288433	4641.183455255787
	4649.805898274172	4571.456815701747	4570.211545611026

Figure 15. Result of the RMSE calculations of each model for subsets number 5 and 10 after removing the outlier

	lasso	reg	ridge
	0.8442037737447713	0.8537742577743399	0.8537137331042383
	0.8392211977078633	0.8445937846437946	0.8446784387756354

Figure 16. Result of the R^2 calculations of each model for subsets number 5 and 10 after removing the outlier

Overall the summary of the evaluation of the three models is provided in Table 2. It can be seen that the Ridge model has the lowest RMSE and highest R^2 (actually slightly higher than no-regularization by 5×10^{-5}).

Table 2. Summary of the evaluation result of the 3 models

	RMSE			R^2		
	No-reg	LASSO	Ridge	No-reg	LASSO	Ridge
Average	4737.6882	4861.3232	4737.1505	0.8291	0.8211	0.8291
St. Dev	143.3280	148.8881	143.7387	0.0434	0.0421	0.0434

5.3. Feature Importance

Due to the One-Hot Encoding process in the featurization step, the number of features increased drastically from 28 to 133, which is considered a lot. Therefore, here, we only listed the top 10 features from each model in Table 3. The complete list of the coefficients is provided in the Jupyter Notebook file.

Table 3. The top 10 features from each model

No-reg	LASSO	Ridge
1. year	1. year	1. year
2. mileage	2. mileage	2. mileage
3. horsepower	3. horsepower	3. horsepower
4. city_fuel_economy	4. torque	4. city_fuel_economy
5. highway_fuel_economy	5. engine_cylinders (V12)	5. highway_fuel_economy
6. torque	6. franchise_make (McLaren)	6. torque
7. engine_cylinders (V12)	7. engine_cylinder (W12 Flex Fuel Vehicle)	7. engine_cylinders (V12)
8. franchise_make (McLaren)	8. franchise_make (Bentley)	8. franchise_make (McLaren)
9. engine_cylinder (W12 Flex Fuel Vehicle)	9. highway_fuel_economy	9. engine_cylinder (W12 Flex Fuel Vehicle)
10. franchise_make (Ferrari)	10. franchise_make (Mercedes-Benz)	10. franchise_make (Ferrari)

From the table above, we can see that the no-regularization and Ridge models have the same top 10 features, whereas LASSO has different ones. Even though the no-regularization and the Ridge models have the same order, the coefficient values are slightly different due to the penalization in the Ridge model. Moreover, we can also see that the top 3 features are always the ‘year’, ‘mileage’, and ‘horsepower’. The ‘year’ column is correlated with the age of the car, where the older the car is, the lower the price should be. On the other hand, the ‘mileage’ column, also has a negative relationship with the price in which higher mileage results in lower price, whereas the ‘horsepower’ column has a positive relationship in which the price of the car will be higher along with how much ‘horsepower’ the car has. Interestingly, the only car manufacturer that consistently made it to the top 10 features is ‘McLaren’. In the no-regularization and the Ridge model, ‘Ferrari’ does come to the top 10 variables, but in the LASSO model ‘Mercedes-Benz’ and ‘Bentley’ come to the top 10 instead. Nevertheless, as we can see the car manufacturers that come to the top 10 are actually the premium car manufacturers.

In addition, we also note that the LASSO model has several features with zero coefficients, whereas the other two models only have near-zero coefficients. This means that in the LASSO model, the unimportant features are explicitly not used by the prediction model. That is the reason why the LASSO model has a slightly higher bias compared with the other two models due to a lower number of predictors (only 79 were used). Therefore, if the goal is to reduce the number of features used, then LASSO is better as it can reduce them significantly.

From the 79 features selected by the LASSO model, we observed that:

- From the 9 body types, only 5 were selected, which are 'Coupe', 'Hatchback', 'Sedan', 'SUV / Crossover', and 'Van'.
- From 26 engine types, only 13 were selected. Those engine types are 'H4', 'I3', 'I4', 'I4 Flex Fuel Vehicle', 'I5', 'I6', 'V6', 'V6 Biodiesel', 'V6 Flex Fuel Vehicle', 'V8 Flex Fuel Vehicle', 'W12', and 'W12 Flex Fuel Vehicle'. Here 'H' stands for the horizontal cylinder type of engine and 'I' stands for the inline cylinder type of engine. From here we can see for inline type all the ones that are made to the model are gasoline engines, except for the 'I4 Flex Fuel Vehicle'. For the 'V6', almost every type of this engine made it into the model except for the 'hybrid' and 'diesel' types.
- From 15 types of listing color, there are 7 that were selected by the LASSO model. Those colors are 'black', 'blue', 'gold', 'red', 'silver', 'white', and 'unknown'. Interestingly, the 'gold' color is pretty significant to the car's price.
- The type of transmission that is only used by the LASSO model is 'CVT'.
- The only type of wheel system that is not used by the LASSO model is '4x2'. This might happen because the other system already made it to the model. Then the last type is considered all zero by the model.
- Of the 47 car manufacturers, there are 29 made it to the models, of which 6 of them are Japanese car manufacturers ('Acura', 'Honda', 'Lexus', 'Mitsubishi', 'Nissan', and 'Toyota'), 2 Korean car manufacturers ('Hyundai' and 'Kia'), 7 US car manufacturers ('Cadillac', 'Chevrolet', 'Chrysler', 'Dodge', 'Ford', 'INFINITI', and 'RAM'), and the rest are European car manufacturers ('Audi', 'Bentley', 'BMW', 'Jaguar', 'Land Rover', 'Lotus', 'Maserati', 'McLaren', 'Mercedes-Benz', 'MINI', 'Porsche', 'Roll-Royce', 'Volkswagen', and 'Volvo').

6. Conclusion

For the models, we can conclude that the Ridge model has a slightly better performance than the other two models in terms of the RMSE and R^2 values. However, the LASSO model can remove unused features and still maintain performance as well as the other models do. Therefore, in term of gaining information about what really impact the price, the LASSO model is the better method.

7. Future Recommendation

In this project, we encountered some limitations, which are:

- The number of records we use is actually far from the number of data sets that come from the source. This limitation is due to removing the missing values.
- We also did not consider the location of the dealer as a predictor.

- We also encountered the extreme price of the car due to the premium car manufacturer or anomaly such as ‘collection’ cars. This kind of record might skew the result of the analysis.
- This project also doesn’t consider in-depth analysis of premium brand or engine preferences for a given type of car, this may give better grouping when predicting the car prices.

Therefore, for future research, we recommend doing these steps to improve the model:

- Do proper missing values imputation to increase the sample size so that the model has better accuracy
- Consider grouping the data based on the location of the dealer
- Consider doing an in-depth analysis of the premium cars or antique collection cars by separating them from the common household cars to avoid extreme prices

8. Work Division

Part	Eddy	Fariz	Robby
1. Project Description	40%	30%	30%
2. Business Model	30%	30%	40%
3. Data			
a. Data Wrangling	40%	30%	30%
b. EDA	30%	30%	40%
4. Machine Learning			
a. Report (theoretical)	35%	35%	30%
b. Jupyter Notebook (coding)	30%	30%	40%
5. Result Discussion & Conclusion	30%	40%	30%
6. Presentation & Video	40%	30%	30%

9. Video Presentation

Youtube link: <https://youtu.be/qoZM5XCVOIY>

Note:

The resources used for this project can be accessed using this link:

https://drive.google.com/drive/folders/1Jlx-MaE2cHIfC8_LHVJs0CiWAGL22cFw?usp=sharing

References

- [1] Khan, S. (2023, July 14). *Americans' dependence on cars is a national security issue*. Inkstick. <https://inkstickmedia.com/americans-dependence-on-cars-is-a-national-security-issue/>
- [2] Wayland, M. (2023, March 25). *Why there may be no return to "normal" for the U.S. used vehicle market*. CNBC. <https://www.cnbc.com/2023/03/25/why-there-may-be-no-return-to-normal-for-the-used-vehicle-market.html>
- [3] Depillis, L., & Smialek, J. (2023, May 20). Why is inflation so stubborn? cars are part of the answer. The New York Times. <https://www.nytimes.com/2023/05/20/business/economy/car-prices-inflation.html>
- [4] CNBC. (2022, January 14). *Soaring used car prices are pushing inflation higher, and there's not much the U.S. can do about it*. CNBC. <https://www.cnbc.com/2022/01/13/why-used-car-prices-are-pushing-inflation-higher.html>
- [5] de Prez, M. (2023, March 24). *Twice as many people have bought a car "unseen" since the pandemic*. AM Online. <https://www.am-online.com/news/used-cars/2023/03/24/twice-as-many-people-buy-a-car-unseen-since-pandemic>
- [6] Thakkar, S., Bhatt, A., Dudhia, K., Vaghela, N., & Kamdar, V. (2023). *International Journal of Research Publication and Reviews*, 4(4), 3151–3155.
- [7] Mammadov, H. (2021). Car Price Prediction in the USA by using Liner Regression. *Journal of Economic Behavior*, 11(1), 99–108. <https://doi.org/10.14276/2285-0430.3049>
- [8] Puteri, C. K., ; Safitri, L. N. (2020). Analysis of linear regression on used car sales in Indonesia. *Journal of Physics: Conference Series*, 1469(1). <https://doi.org/10.1088/1742-6596/1469/1/012143>
- [9] Demir-Kavuk, O., Kamada, M., Akutsu, T., & Knapp, E.-W. (2011). Prediction using step-wise L1, L2 regularization and feature selection for small data sets with large number of features. *BMC Bioinformatics*, 12(1). <https://doi.org/10.1186/1471-2105-12-412>